

FORMATION EMBARQUÉE

TP — tp5 stm32 freertos

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 5 — Station météo STM32 + FreeRTOS (≈ 10 h, en 4 séances)

Objectif pédagogique : porter la station météo du TP 1 sur un environnement professionnel — Nucleo, HAL, drivers écrits maison, DMA, puis architecture multi-tâches FreeRTOS. À la fin, tu as le projet « candidature firmware junior » type.

 **Fiches minutées séance par séance** : [Séance 1 — Socle](#) · [Séance 2 — Driver BME280](#) · [Séance 3 — ADC/DMA + console](#) · [Séance 4 — FreeRTOS](#).

Matériel

- Nucleo-F411RE (ou toute Nucleo-64 ; adapter les broches).
- Capteur **BME280** (I2C — température/humidité/pression, mieux documenté que le DHT22 pour écrire un driver propre) ; à défaut, un potentiomètre simulera la « température ».
- STM32CubeIDE installé. Prérequis : module 10 + TD 10 faits.

Discipline du TP : un commit Git par étape verte ; les drivers dans `Core/Src/drivers/` avec leurs `.h` — le `main.c` ne contient jamais d'accès direct à un capteur.

Séance 1 (2 h) — Socle : horloges, UART de debug, GPIO

1. Projet CubeMX pour la carte (Board Selector → périphériques par défaut). Clock tree : SYSCLK à 100 MHz par PLL depuis HSI — note dans ton compte rendu les valeurs M/N/P choisies par l'outil et vérifie $APB1 \leq 50$ MHz.
2. `printf` redirigé vers USART2 (TD 10 ex. 2). Bannière de démarrage avec version (`__DATE__` , `__TIME__`) — réflexe pro : savoir QUEL firmware tourne.
3. LED LD2 en vie (« heartbeat ») : clignotement 1 Hz cadencé par `HAL_GetTick()` (pas de `HAL_Delay` : le socle doit déjà être non bloquant). Bouton B1 par EXTI → inverse la verbosité des logs.

✓ **Point de contrôle 1** : bannière au terminal série, heartbeat régulier, appui bouton pris en compte — et tu sais dire à quelle fréquence tourne chaque bus.

Séance 2 (3 h) — Driver BME280 écrit maison

Interdiction d'utiliser une bibliothèque toute faite : le but EST le driver.

Étape 2.1 — Reconnaissance

I2C1 (PB8/PB9) à 100 kHz. Adresse BME280 : 0x76 ou 0x77 selon câblage — écris d'abord un **scanner I2C** :

```
for (uint8_t a = 1; a < 128; a++)
    if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 1, 5) == HAL_OK)
        printf("trouve : 0x%02X\r\n", a);
```

Lis le registre `id` (0xD0) : il doit répondre **0x60**. C'est ton « hello world » I2C.

Étape 2.2 — Le driver

Interface imposée (`drivers/bme280.h`) :

```
typedef struct {
    I2C_HandleTypeDef *i2c;
    uint8_t adresse;           // déjà décalée <<1
    // étalonnage lu en NVM du capteur (datasheet §4.2.2)
    uint16_t dig_T1; int16_t dig_T2, dig_T3;
    /* ... dig_P*, dig_H* ... */
} bme280_t;

bool bme280_init(bme280_t *dev, I2C_HandleTypeDef *i2c, uint8_t addr7);
bool bme280_lire(bme280_t *dev, int32_t *temp_centiemes,
                uint32_t *press_pa, uint32_t *hum_milliemes);
```

Étapes guidées (datasheet Bosch BME280 ouvert à côté) : 1. `init` : vérifier l'id, lire les registres d'étalonnage (0x88..., 0xE1...), configurer `ctrl_hum`, `ctrl_meas`, `config` (oversampling ×1, mode normal). 2. `lire` : lecture **en rafale** des 8 octets 0xF7-0xFE (une seule transaction — les valeurs sont cohérentes entre elles), puis les formules de **compensation en entiers** de la datasheet (§4.2.3 — les recopier est normal, les comprendre est demandé). 3. Sorties en **point fixe** : centièmes de °C, Pa, millièmes de % — aucun `float` dans le driver (module 01 §2.1). 4. Chaque fonction retourne `false` sur échec HAL : le driver ne fait jamais de `printf` et ne bloque jamais plus que ses timeouts I2C.

✓ **Point de contrôle 2** : `T=23.45C P=101325Pa H=48.2%` plausibles au terminal, ET débrancher SDA en cours de route produit des `false` gérés (messages d'erreur côté main, pas de gel).

Séance 3 (2 h) — ADC + DMA et console

1. **ADC1 + DMA circulaire** sur 2 canaux (potentiomètre PA0 = consigne, capteur de température interne du STM32 en canal 16 — comparaison amusante avec le BME280) : tableau `uint16_t adc[2]` rempli en tâche de fond (TD 10 / module 10 §4.6). Moyenne glissante sur 16 échantillons.
2. **Console UART** (reprise du TD 10 ex. 2) avec commandes : `status` (mesures + uptime + version), `log on|off`, `seuil <valeur>`.

✓ **Point de contrôle 3** : pose un point d'arrêt et vérifie dans la vue mémoire que `adc[]` continue de changer pendant que le CPU est arrêté — c'est le DMA, et savoir le *montrer* est le but de l'étape.

Séance 4 (3 h) — Architecture FreeRTOS

Active FreeRTOS (CMSIS-V2) dans CubeMX (base de temps HAL → TIM10 quand l'outil le demande — note pourquoi : SysTick appartient à l'OS).

Architecture imposée

Tâche	Priorité	Période	Rôle
T_Capteur	normale	1 s	lit le BME280, pousse une <code>mesure_t</code> dans <code>q_mesures</code>
T_Traitement	normale	sur message	moyennes, comparaison seuil, pousse vers <code>q_affichage</code>
T_Console	basse	sur caractère	la console série (bloquée sur la queue RX alimentée par l'ISR UART)
T_Heartbeat	la plus basse	500 ms	LED vie + surveillance des piles (<code>uxTaskGetStackHighWaterMark</code>)

Règles : - **Toute communication par queue** (`osMessageQueue`) — aucune variable globale partagée entre tâches (le mutex n'apparaît que si tu partages l'I2C entre deux tâches : explique pourquoi tu ne le fais pas). - L'ISR UART ne fait que pousser l'octet dans une queue (`...FromISR`) — vérifie la priorité NVIC compatible FreeRTOS (module 10 §4.7). - `configCHECK_FOR_STACK_OVERFLOW = 2` pendant la mise au point, et le hook qui `printf` + LED rouge.

Essais de validation

#	Essai	Attendu
1	Fonctionnement 10 min	mesures stables, aucune dérive des piles (high water marks stables)
2	Débrancher le capteur en marche	<code>T_Capteur</code> signale, les autres tâches vivent, reprise au rebranchement
3	Spam de la console (coller 1 Ko de texte)	pas de crash, pas d'octets mélangés dans les logs
4	Point d'arrêt dans <code>T_Traitement</code> 5 s puis reprise	le système rattrape : messages en attente traités, pas de perte silencieuse non expliquée

✓ **Point de contrôle final** : pour l'essai 4, explique dans le compte rendu ce que sont devenus les messages produits pendant la pause (queue pleine ? politique ?) — il n'y a pas de « bonne » réponse unique, il y a une réponse **documentée**.

Livrables et barème

Livrable	Points
Socle : clock tree expliqué, printf, heartbeat non bloquant	/3
Driver BME280 : compensation entière, erreurs gérées, zéro dépendance	/6
ADC+DMA démontré (mesure sous point d'arrêt) + console robuste	/4
Architecture FreeRTOS conforme (queues, ISR minimales, priorités justifiées)	/4
Les 4 essais documentés + README de projet avec schéma	/3
Total	/20

Après ce TP : ton dépôt contient un driver I2C bare-metal, du DMA, une architecture RTOS propre et des essais documentés — mets-le en tête de ton portfolio, c'est exactement ce qu'un recruteur firmware ouvre en premier.